

JAVA - שיעור 7

נושא 1: שימוש במילת המפתח this, פירוק מתודות ומניעת

כפילויות קוד

הסבר מקוצר:

מילת המפתח this משמשת כהפניה לאובייקט הנוכחי ופותרת התנגשויות שמיות בין תכונות לפרמטרים מקומיים. פירוק מתודות מורכבות לתתי-מתודות (כולל מתודות פרטיות private) מונע כפילות קוד ומשפר את התחזוקה.

הסבר מורחב:

כאשר ישנו פרמטר במתודה או בקונסטרוקטור בעל שם זהה לתכונה של המחלקה, נוצרת התנגשות שמות (Shadowing). השימוש בביטוי this.length מבהיר למהדר כי הכוונה היא לתכונה של האובייקט הנוכחי שעליו הופעלה הפעולה ברגע הנתון, בעוד שציון השם length לבדו פונה לפרמטר המקומי. אף על פי שניתן להשתמש ב-this גם ללא התנגשות שמות לצורך קריאות, במקרים של התנגשות שמיות השימוש בו הוא חובה.

בנוסף, מתודות ארוכות המבצעות מספר פעולות שונות (כמו חישוב אלכסון המלבן, חלוקתו בשתיים לצורך מציאת רדיוס ויצירת מעגל חדש) הן קשות לתחזוקה ולהבנה. פירוקן לתתי-מתודות קטנות וממוקדות – למשל, הוצאת חישוב האלכסון למתודה ייעודית בשם computeDiagonal – מאפשר שימוש חוזר בקוד, מניעת כפילויות, ועדכון קל של נוסחאות במקום יחיד בעתיד. אם המתודה מיועדת לחישוב פנימי בלבד ואינה נחוצה למשתמשי המחלקה בחוץ, מומלץ להגדירה תחת הרשאת גישה private ובכך לשמור על עיקרון הסתרת המימוש.

משפטי מפתח והגדרות למבחן:

- **מילת המפתח this:** הפנייה (Reference) המצביעה על האובייקט הנוכחי שעליו מופעלת המתודה ברגע הנתון.
- **כפילות קוד (Code Duplication):** כתיבת אותה לוגיקה או נוסחה במספר מקומות שונים. זוהי פרקטיקה גרועה המקשה על תחזוקה ויוצרת באגים.
- **הרשאת גישה private למתודות:** הגבלת מתודה לשימוש פנימי בלבד בתוך אותה מחלקה, במטרה להסתיר את פרטי המימוש הפנימיים מהמשתמש החיצוני.

דוגמאות:

- פתרון התנגשות שמות בקונסטרוקטור של מלבן:

```
public Rectangle(double length, double width) {
    this.length = length;
    this.width = width;
}
```
- וקריאה לה private double computeDiagonal() פירוק מתודה לחישוב אלכסון המלבן: יצירת מתודה במקום לבצע את חישוב שורש פיתגורס ישירות בתוכה getBoundingCircle מתוך:

```
private double computeDiagonal() {
    return Math.sqrt(this.length * this.length + this.width * this.width);
}
```

}

שאלות הבנה:

1. מתי חובה להשתמש במילת המפתח `this` ומתי השימוש בה הוא אופציונלי בלבד?
2. מהם היתרונות של פירוק מתודה ארוכה המבצעת מספר חישובים שונים לכמה מתודות קטנות יותר?
3. אם נגדיר מתודת עזר כ-`private`, האם נוכל לקרוא לה מתוך מתודת ה-`main` של התוכנית? הסבר מדוע.

נושא 2: השוואת אובייקטים (כתובות לעומת ערכים) ומימוש השוואות לוגיות

הסבר מקוצר:

אופרטור ההשוואה `==` ב-Java משווה את כתובות הזיכרון (ההפניות) של האובייקטים ולא את תוכנם. כדי לבצע השוואה תוכנית אמיתית לפי ערכים, יש לממש מתודות השוואה ייעודיות כמו `isEqual`.

הסבר מורחב:

ב-Java, משתנה המייצג אובייקט אינו מכיל את האובייקט עצמו אלא הפניה (Reference) – כתובת בזיכרון המצביעה על מיקום האובייקט. כאשר משתמשים באופרטור `==` בין שני משתני אובייקט, המהדר בודק האם שתי ההפניות מצביעות על אותה כתובת זיכרון פיזית בדיוק. אם יצרנו שני מלבנים שונים בעזרת המילה `new` בעלי אותן תכונות בדיוק, הם יישמרו במיקומים שונים בזיכרון וההשוואה `==` תחזיר `(false)`.

כדי להשוות את הערכים הפנימיים של האובייקטים, אנו מגדירים מתודות השוואה ייעודיות כמו `isEqual(Rectangle other)` הבודקות האם התכונות (האורך והרוחב) של האובייקט הנוכחי (`this`) שוות לאלו של האובייקט האחר שהתקבל כפרמטר (`other`). באופן דומה, ניתן לממש מתודות כגון `isBigger` ו-`isSmaller` המגדירות את חוקי ההשוואה המדויקים (למשל השוואה לפי שטח או היקף). מימוש חכם של `isSmaller` יכול להתבצע פשוט על ידי קריאה הפוכה ל-`isBigger` (`this.isBigger(other)`), מה שמונע שכפול מיותר של קוד החישוב ומקל על שינוי הלוגיקה בעתיד.

משפטי מפתח והגדרות למבחן:

- **השוואת הפניות (Reference Equality):** שימוש באופרטור `==` הבודק האם שני משתנים מצביעים על אותה כתובת פיזית בזיכרון.
- **השוואה לוגית (Logical Equality):** השוואת הערכים של תכונות האובייקט באמצעות מתודה מותאמת אישית שהוגדרה על ידי המתכנת.
- **מניעת כפל מימוש בהשוואות:** מימוש מתודה אחת (כמו `isSmaller`) על בסיס היפוך הלוגיקה של מתודה קיימת (כמו `isBigger`), כדי לרכז את חוקיות ההשוואה במקום אחד.

דוגמאות:

- השוואה שגויה של מלבנים זהים בזיכרון:

```
Rectangle r1 = new Rectangle(100, 25);  
Rectangle r2 = new Rectangle(100, 25);  
// r1 == r2 יחזיר false שערך זהים
```
- מימוש השוואה לוגית קצרה:

```
public boolean isEqual(Rectangle other) {  
    return this.length == other.length && this.width == other.width;  
}
```
- ללא כפילות קוד `isSmaller` ליצור מימוש `isBigger`-שימוש ב:

```
public boolean isSmaller(Rectangle other) {
```

```
    return other.isBigger(this);  
}
```

שאלות הבנה:

1. מדוע השוואה של שני אובייקטים שנבנו באמצעות המילה new בעלי ערכים זהים לחלוטין מחזירה false כאשר משתמשים באופרטור ==?
2. כיצד ניתן לממש את המתודה isSmaller בצורה חכמה שמונעת שכפול של לוגיקת ההשוואה שנכתבה כבר ב-isBigger?
3. האם מותר למתכנת להחליט שהשוואה בין שני מלבנים (isEqual) תתבצע על בסיס שטח המלבן בלבד, גם אם אורכי הצלעות שונים? מה המשמעות של החלטה כזו?

נושא 3: מתודת toString כמנגנון ייצוג ותצוגה סטנדרטי מובנה

הסבר מקוצר:

מתודת toString היא מתודה מובנית וסטנדרטית ב-Java המחזירה ייצוג טקסטואלי של האובייקט. השפה מפעילה אותה באופן אוטומטי בעת ניסיון להדפיס את האובייקט או בעת שרשורו למחרוזת אחרת.

הסבר מורחב:

בעבר השתמשנו במתודות תצוגה מותאמות אישית כמו display(), אשר הדפיסו ישירות למסך את ערכי האובייקט אך לא החזירו ערך (הוגדרו כ-void). חסרונה המרכזי של גישה זו הוא חוסר הגמישות – המתכנת המשתמש במחלקה אינו יכול לשלוט בקלות ביעד ההדפסה, לשמור את המידע בקובץ, או לשלב אותו בתוך טקסט מורכב יותר.

במקום זאת, הסטנדרט המקובל ב-Java הוא לממש מתודה ציבורית בשם toString המוגדרת כ-public String toString(). מתודה זו אינה מדפיסה דבר בעצמה, אלא בונה ומחזירה אובייקט String המכיל את המידע המעוצב על האובייקט. היתרון העצום שלה הוא ש-Java מזהה אותה באופן מובנה בשפה: בכל פעם שנבצע הדפסה של האובייקט ישירות (למשל System.out.println(acc1)) או נשרשר אותו למחרוזת (למשל "Owner: " + acc1), המערכת תקרא אוטומטית ובאופן סמוי ל-toString() ותשתול את הטקסט המוחזר. זה הופך את מתודת display() הישנה למיותרת לחלוטין.

משפטי מפתח והגדרות למבחן:

- **מתודת toString:** מתודה מובנית בשפה בעלת החתימה public String toString(), שמטרתה לספק ייצוג טקסטואלי רשמי של תוכן האובייקט.
- **קריאה סמויה (Implicit Invocation):** מנגנון שבו Java קוראת בעצמה ל-toString() כאשר אובייקט משורשר למחרוזת או מועבר כפרמטר לפונקציית הדפסה, ללא צורך בכתיבה מפורשת של שם המתודה בקוד.

דוגמאות:

- עבור חשבון בנק toString מימוש מתודת:

```
public String toString() {  
    return "Cash account - Number: " + this.number + ", Balance: " + this.balance;  
}
```
- (שרשור אוטומטי של אובייקט (הפעלה סמויה):

```
CashAccount acc1 = new CashAccount(1, 9000.0, owner1);  
System.out.println(acc1); // ידפיס את תוצאתה acc1.toString() - יקרא אוטומטית ל
```

שאלות הבנה:

1. מהם היתרונות של שימוש במתודת toString() על פני מימוש מתודת display() המדפיסה בעצמה למסך?
2. מה יקרה אם ננסה להדפיס אובייקט של מחלקה שבה לא מימשנו כלל את מתודת toString()??
3. האם חובה לכתוב באופן מפורש acc1.toString() בתוך פקודת הדפסה כדי שהמחרוזת שלו תודפס?

הסבר את מנגנון הקריאה הסמויה.

נושא 4: הרכבת אובייקטים (Object Composition) ועיקרון הכמיסה (Encapsulation)

הסבר מקוצר:

הרכבת אובייקטים (Composition) היא תבנית שבה אובייקט אחד מכיל הפניות לאובייקטים אחרים כתכונות פנימיות שלו. שמירה על התכונות כ-private ובחירה קפדנית של מתודות get ו-set מגנות על שלמות האובייקט ומונעות שינויים לא חוקיים.

הסבר מורחב:

בעולם האמיתי, מערכות תוכנה מורכבות אינן מיוצגות רק על ידי טיפוסים פרימיטיביים (כמו מספרים או מחרוזות), אלא מאובייקטים המכילים אובייקטים אחרים. קשר זה נקרא קשר "Has-A" (יש לו). לדוגמה, לחשבון בנק (CashAccount) "יש" בעלים (AccountOwner), שהוא בעצמו אובייקט בעל תעודת זהות, שם, טלפון ואימייל. במקרה זה, אחת התכונות במחלקה CashAccount תהיה מטיפוס AccountOwner. בזיכרון המחשב, משתנה החשבון מכיל הפניה (כתובת) לאובייקט החשבון, והתכונה owner שבתוך החשבון מכילה הפניה נוספת המצביעה על אובייקט הבעלים המנוהל בזיכרון.

על מנת להבטיח את עקרון הכמיסה (Encapsulation), חובה להגדיר את כל התכונות כ-private. הגישה אליהן צריכה להתבצע דרך מתודות ציבוריות באופן מבוקר. המתכנת מפעיל שיקול דעת אילו מתודות שינוי (set) לחשוף: למשל, אין זה הגיוני לאפשר שינוי של תעודת זהות של בעל חשבון (setID) או להחליף בעלים לחשבון קיים (setOwner), ולכן לא נממש מתודות אלו כלל. בנוסף, מתודות שינוי ופעולות לוגיות (כמו משיכה או הפקדה) חייבות להכיל בדיקות תקינות קפדניות (ולידציה) ולזרוק שגיאות (למשל IllegalArgumentException) כדי למנוע מצבים לא חוקיים כמו הפקדת סכום שלילי או כניסה ליתרה שלילית בחשבון מזומן.

משפטי מפתח והגדרות למבחן:

- **הרכבת אובייקטים (Object Composition):** תכנון מחלקות שבהן לפחות אחת מהתכונות היא אובייקט ממחלקה אחרת, המייצג קשר הכלה ("Has-A").
- **כמיסה סלקטיבית (Selective Encapsulation):** בחירה מודעת של המתכנת אילו פעולות לחשוף החוצה (ב-public) ואילו להסתיר, כדי למנוע שינויים שלא תואמים את הלוגיקה העסקית של המערכת.
- **ולידציה (בדיקת תקינות נתונים):** מנגנוני בדיקה פנימיים (באמצעות if וזריקת חריגות) המבטיחים שהאובייקט תמיד יישאר במצב חוקי ותקין.

דוגמאות:

- AccountOwner המכילה תכונה פנימית מטיפוס CashAccount הגדרת הרכבה: מחלקת:

```
public class CashAccount {
    private int number;
    private double balance;
    private AccountOwner owner; // הרכבת אובייקטים
}
```
- (בדיקת תקינות במשיכת כסף (מניעת יתרה שלילית):

```
if (amount > 0 && this.balance - amount >= 0) {
```

```

        this.balance -= amount;
    } else {
        System.out.println("Current balance not allowing withdraw");
    }
}

```

- main-שינוי תכונה של אובייקט פנימי באמצעות שרשור מתודות ב
 עדכון שם משפחת הבעלים של חשבון 1 // acc1.getOwner().setLastName("Cohen");

שאלות הבנה:

1. מהו קשר "Has-A" וכיצד הוא בא לידי ביטוי בהגדרת התכונות של מחלקת CashAccount?
2. אם התכונה owner מוגדרת כ-private בתוך CashAccount, כיצד נוכל בכל זאת לשנות את שם המשפחה של בעל החשבון מתוך ה-main?
3. מדוע לא נכון לממש מתודות set עבור כל תכונה ותכונה במחלקה? הבא שתי דוגמאות שבהן היעדר מתודת set שומר על שלמות הלוגיקה של המערכת.

טבלה מסכמת של הנושאים

נושא	הגדרות והרחבות מסכמות
מילת המפתח this ופירוק מתודות	מילת המפתח this מייצגת את האובייקט הנוכחי. פירוק מתודות מורכבות למתודות פשוטות יותר מונע כפל קוד, כאשר הרשאת private משמשת להסתרת לוגיקה פנימית בלבד.
השוואת אובייקטים	אופרטור == בודק שוויון של כתובות זיכרון (הפניות). השוואה של תוכן אובייקטים (שוויון לוגי) דורשת מאיתנו לממש מתודה ייעודית כמו isEqual.
מתודת toString	החלופה המודרנית והסטנדרטית ל-display(). מחזירה ייצוג טקסטואלי של האובייקט כ-String ונקראת באופן סמוי בעת הדפסה או שרשור.
הרכבת אובייקטים ועיקרון הכמיסה	בניית מחלקה המכילה תכונה שהיא בעצמה אובייקט ממחלקה אחרת. יש להקפיד על הגדרת התכונות כ-private ולבצע בדיקות ולידציה קפדניות.

סיכום פקודות ופונקציות (Java)

שם הפקודה/פונקציה	כיצד להשתמש	מה היא עושה	דוגמה קצרה
this	מצטרף לתכונות או מתודות בתוך המחלקה	מצביע על האובייקט הנוכחי ופותר התנגשויות שמיות מול פרמטרים	<code>this.number = ;number</code>
private	לפני הגדרת תכונה או מתודה במחלקה	מגביל את הגישה אליהן כך שניתן לגשת אליהן רק מתוך המחלקה עצמה	<code>private AccountOwner ;owner</code>
public	לפני הגדרת מחלקה, קונסטרקטור או מתודה	מאפשר גישה מלאה וקריאה לרכיב מכל מקום בתוכנית	<code>public double ()getBalance</code>
toString	פונקציה המחזירה String ללא פרמטרים	בונה ייצוג טקסטואלי של האובייקט שמופעל סמויה בהדפסות ושרשורים	<code>public String toString() { return { ;"ID: " + id</code>
throw new	בתוך בדיקות תקינות במתודות או קונסטרקטורים	קוטע מיידית את ריצת התוכנית ומדווח על שגיאת קלט לא חוקית	<code>throw new IllegalArgumentException("Invalid ;("amount</code>

נספח: תשובות לשאלות ההבנה

תשובות לנושא 1:

1. חובה להשתמש במילת המפתח `this` כאשר ישנו פרמטר או משתנה מקומי במתודה ששמו זהה לחלוטין לשמה של תכונה במחלקה (מצב של התנגשות שמיות). השימוש ב-`this` הוא אופציונלי אך מומלץ כאשר אין התנגשות שמות, במטרה לשפר את קריאות הקוד ולהבהיר שמדובר בפנייה לתכונת האובייקט.
2. פירוק מתודה ארוכה לכמה מתודות קטנות וממוקדות מקל על הבנת הקוד, מאפשר בדיקה (Testing) פשוטה יותר של כל יחידת קוד, מונע כפילות קוד (Code Duplication) ומאפשר שימוש חוזר באותם חישובים במקומות נוספים במחלקה. בנוסף, אם נוסחת החישוב תשתנה בעתיד, נצטרך לתקן אותה רק במתודה המפורקת ולא במספר מקומות.
3. לא. מתודת `עזר` שהוגדרה כ-`private` חסומה לחלוטין לגישה מחוץ למחלקה שבה היא נכתבה. מתודת ה-`main` של התוכנית, הנמצאת במחלקה נפרדת (כמו `BankProgram`), לא תוכל להפעיל אותה, וניסיון כזה יגרום לשגיאת קומפילציה.

תשובות לנושא 2:

1. המילה `new` מקצה באופן מפורש מקום חדש בזיכרון עבור האובייקט. לכן, שני אובייקטים שנבנו בנפרד מקבלים שתי כתובות זיכרון שונות לחלוטין. מאחר ואופרטור `==` משווה את כתובות הזיכרון (ההפניות) ולא את ערכי התכונות, ההשוואה תחזיר `false` למרות שהתכונות הפנימיות זהות.
2. ניתן לממש את `isSmaller` על ידי קריאה ל-`isBigger` מהאובייקט השני שהתקבל כפרמטר (`other`), תוך שליחת האובייקט הנוכחי (`this`) כפרמטר אליו. קוד המקור ייראה כך: `return other.isBigger(this);`. בדרך זו, הלוגיקה והנוסחה של ההשוואה נשמרות במקום אחד בלבד.
3. כן, מותר למתכנת להחליט זאת. אין חוק קשיח בשפה שמכתיב מהו שוויון לוגי, והמתכנת הוא זה שקובע את קריטריוני ההשוואה בתוך המתודה `isEqual`. המשמעות היא ששני מלבנים בעלי מימדים שונים לחלוטין (למשל מלבן של `2x3` ומלבן של `1x6`) ייחשבו "שווים" לוגית בעיני התוכנית שלנו מכיוון שהשטח של שניהם זהה (6).

תשובות לנושא 3:

1. מתודת `display()` הנה מתודה מסוג `void` המדפיסה באופן קשיח ישירות לתוך פלט המערכת (`System.out`). לעומת זאת, `toString()` מחזירה את המידע כמחרוזת (`String`), מה שמעניק גמישות עצומה: המתכנת המפעיל יכול להחליט לשמור את המידע בקובץ, לשלוח אותו ברשת, להציג אותו בתיבת דיאלוג גרפית, או לשרשר אותו להודעה ארוכה יותר לפני ההדפסה.
2. המהדר לא ייכשל בקומפילציה, אך בעת הדפסת האובייקט `Java` תפעיל את מתודת ברירת המחדל של מחלקת העל `Object`. הדפסה זו תציג פלט "ג'בריש" הכולל את שם המחלקה, סימן `@` וקוד ה-`Hash` של האובייקט (המבוסס על כתובתו בזיכרון).
3. לא, אין חובה לכתוב זאת באופן מפורש. בשל מנגנון הקריאה הסמויה של השפה, בכל פעם שאנו מוסרים אובייקט לפעולת הדפסה (כמו `System.out.println(acc1)`) או משרשרים אותו למחרוזת אחרת, `Java` מזהה בעצמה את האובייקט ומפעילה מאחורי הקלעים את מתודת ה-`toString()` שלו באופן אוטומטי.

תשובות לנושא 4:

1. קשר "Has-A" (יש לו) מתאר הכלה של אובייקט אחד בתוך אובייקט אחר. בהגדרת מחלקת

CashAccount, קשר זה בא לידי ביטוי בכך שאחת התכונות (Attributes) של החשבון מוגדרת מטיפוס המחלקה AccountOwner (private AccountOwner owner);, כלומר, לכל חשבון בנק "יש" אובייקט בעלים.

2. אף על פי שהתכונה owner מוגדרת כ-private ואינה נגישה ישירות, אנו משתמשים במתודת ה-get הציבורית שלה (getOwner()) כדי לקבל את הפניית הבעלים, ועל גביה מפעילים בשירשור את מתודת ה-set של הבעלים (setLastName). הקוד יכתב כך: acc1.getOwner().setLastName("Cohen");. מתודת set חושפת את התכונה לשינוי חופשי מבחוץ, דבר שעלול לפגוע בלוגיקה העסקית ובחוקיות האובייקט. דוגמה 1: לא נרצה לממש setID עבור בעל חשבון כי תעודת זהות אינה משתנה במהלך החיים. דוגמה 2: לא נרצה לממש setOwner בחשבון בנק כי בעלות על חשבון קבוע אינה ניתנת להחלפה ישירה באופן שרירותי (יש לסגור חשבון ולפתוח חדש).